

RÉSILIENCE 976

Guide complet de création du MVP avec Vue.js

Une seule application front-end - sans compte - sans base de données - mobile-first

Décision	Choix retenu
Framework	Vue 3 + Vite + TypeScript
Architecture	Application statique SPA/PWA légère
Comptes utilisateurs	Aucun
Back-end	Aucun dans le MVP
Stockage	localStorage dans le navigateur
Contenus	Fichiers JSON versionnés
PDF	Génération côté navigateur
Hébergement	Netlify, Vercel, Cloudflare Pages ou OVH statique

Objectif : permettre à un utilisateur de réaliser un diagnostic en 5 à 10 minutes, obtenir un score, un plan d'actions, une checklist, un kit d'urgence et un PDF personnalisé.

Version 1.0 - 3 août 2026

1. Vision et périmètre simplifié

Ce guide transforme le book produit Résilience 976 en un MVP réellement simple à développer seul avec Vue.js. Le projet conserve le cœur utile du produit : diagnostic, scoring, recommandations, checklist, kit d'urgence et export PDF. Les fonctions qui nécessitent des comptes, une base de données, des rôles, une modération ou une infrastructure IA sont volontairement exclues du premier lancement.

1.1 Parcours utilisateur final

1. Arriver sur la page d'accueil et comprendre la promesse.
2. Démarrer le diagnostic sans créer de compte.
3. Répondre à 20 à 30 questions réparties par domaine.
4. Obtenir un score global et des scores par domaine.
5. Voir trois actions immédiates et cinq actions à faire sous sept jours.
6. Consulter une checklist et un kit adaptés au foyer.
7. Télécharger un PDF récapitulatif.
8. Reprendre plus tard sur le même appareil grâce au stockage local.

1.2 Fonctionnalités incluses

- Accueil et présentation du service
- Diagnostic anonyme
- Sauvegarde locale automatique
- Calcul du score dans le navigateur
- Recommandations personnalisées
- Checklist interactive
- Générateur de kit d'urgence
- Résultats par domaine
- Export PDF
- Fiches de ressources officielles
- Quiz simple optionnel
- Pages légales et sources

1.3 Fonctionnalités exclues du MVP

- Création de compte et authentification
- Base de données serveur
- Back-office et CMS
- Campagnes scolaires et classements
- Assistant IA génératif
- Cartographie collaborative
- Signalements publics
- Notifications push
- Application mobile native
- Statistiques individuelles

2. Plan général de réalisation

Phase	Objectif	Livrable
0. Cadrage	Figer les contenus, le périmètre et les règles de score	Dossier de cadrage validé
1. Initialisation	Créer le projet Vue et l'outillage	Projet exécutable en local
2. Structure	Mettre en place routes, stores, layout et design system	Squelette navigable
3. Données	Créer les JSON questions, réponses, actions et sources	Base éditoriale locale
4. Diagnostic	Construire le questionnaire et la sauvegarde locale	Diagnostic terminé sans compte
5. Scoring	Calculer les scores et priorités	Résultats reproductibles
6. Plan et kit	Générer les recommandations, checklist et kit	Parcours utile complet
7. PDF	Produire un document imprimable côté client	PDF téléchargeable

Phase	Objectif	Livrable
8. Qualité	Tester mobile, accessibilité, performance et sécurité	Version candidate
9. Déploiement	Publier et documenter	MVP en production

3. Préparer l'environnement de développement

3.1 Prérequis

- Node.js LTS récent
- npm ou pnpm
- Git
- Visual Studio Code
- Navigateur Chrome et Firefox
- Compte GitHub ou GitLab
- Compte d'hébergement statique

3.2 Création du projet

```
npm create vite@latest resilience-976 -- --template vue-ts
cd resilience-976
npm install
npm install vue-router pinia
npm install jspdf html2canvas
npm install zod
npm install -D vitest @vue/test-utils jsdom
npm run dev
```

TypeScript est conseillé pour sécuriser les structures de données, les calculs et la maintenance. Le projet peut être réalisé en JavaScript, mais les exemples de ce guide utilisent TypeScript.

3.3 Dépendances recommandées

Paquet	Utilité	Obligatoire
vue-router	Navigation entre les pages	Oui
pinia	État du diagnostic et persistance	Oui
jspdf	Génération du PDF	Oui
html2canvas	Capture de blocs visuels dans le PDF	Optionnel
zod	Validation des fichiers JSON	Recommandé
vitest	Tests unitaires	Recommandé
@vue/test-utils	Tests des composants	Recommandé

4. Arborescence du projet

```
resilience-976/
├── public/
│   ├── icons/
│   ├── images/
│   └── documents/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── AppHeader.vue
│   │   ├── AppFooter.vue
│   │   ├── ProgressBar.vue
│   │   ├── QuestionCard.vue
│   │   ├── ScoreGauge.vue
│   │   ├── ActionCard.vue
│   │   └── SourceLink.vue
│   └── data/
│       ├── questions.json
│       ├── actions.json
│       ├── kit.json
│       └── resources.json
```

```

├── sources.json
├── models/
│   ├── assessment.ts
│   ├── question.ts
│   └── recommendation.ts
├── router/
│   └── index.ts
├── services/
│   ├── scoring.service.ts
│   ├── recommendation.service.ts
│   ├── storage.service.ts
│   └── pdf.service.ts
├── stores/
│   └── assessment.store.ts
├── views/
│   ├── HomeView.vue
│   ├── DiagnosticView.vue
│   ├── ResultsView.vue
│   ├── ChecklistView.vue
│   ├── KitView.vue
│   ├── ResourcesView.vue
│   ├── LegalView.vue
│   └── NotFoundView.vue
├── App.vue
├── main.ts
├── tests/
└── package.json

```

5. Routes et pages

Route	Vue	Rôle
/	HomeView	Présenter le service et lancer le diagnostic
/diagnostic	DiagnosticView	Questionnaire pas à pas
/resultats	ResultsView	Score, domaines et priorités
/checklist	ChecklistView	Liste interactive
/kit	KitView	Kit personnalisé
/ressources	ResourcesView	Fiches et liens officiels
/mentions-legales	LegalView	Informations légales et responsabilité
/:pathMatch(.*)*	NotFoundView	Page 404

```

import { createRouter, createWebHistory } from 'vue-router'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: () => import('@views/HomeView.vue') },
    { path: '/diagnostic', component: () => import('@views/DiagnosticView.vue') },
    { path: '/resultats', component: () => import('@views/ResultsView.vue') },
    { path: '/checklist', component: () => import('@views/ChecklistView.vue') },
    { path: '/kit', component: () => import('@views/KitView.vue') },
    { path: '/ressources', component: () => import('@views/ResourcesView.vue') },
    { path: '/mentions-legales', component: () => import('@views/LegalView.vue') },
    { path: '/*', component: () => import('@views/NotFoundView.vue') }
  ],
  scrollBehavior: () => ({ top: 0 })
})

export default router

```

6. Modèles de données

6.1 Type Question

```
export type Criticality = 'low' | 'medium' | 'high' | 'critical'
```

```
export interface AnswerOption {
  id: string
  label: string
  score: number
}

export interface Question {
  id: string
  domain: string
  text: string
  help?: string
  required: boolean
  weight: number
  criticality: Criticality
  answers: AnswerOption[]
  actionIds: string[]
  sourceIds: string[]
}
```

6.2 Exemple questions.json

```
[
  {
    "id": "water_01",
    "domain": "water_food",
    "text": "Disposez-vous d'une réserve d'eau adaptée à votre foyer ?",
    "required": true,
    "weight": 5,
    "criticality": "critical",
    "answers": [
      { "id": "none", "label": "Non", "score": 0 },
      { "id": "partial", "label": "Partiellement", "score": 50 },
      { "id": "ready", "label": "Oui", "score": 100 }
    ],
    "actionIds": ["action_water_reserve"],
    "sourceIds": ["source_water_01"]
  }
]
```

6.3 Exemple actions.json

```
[
  {
    "id": "action_water_reserve",
    "title": "Préparer une réserve d'eau",
    "priorityBand": "now",
    "effort": "medium",
    "why": "Une coupure d'eau peut survenir après un événement majeur.",
    "instructions": [
      "Vérifier la recommandation officielle en vigueur",
      "Adapter la quantité à la composition du foyer",
      "Contrôler régulièrement la rotation des réserves"
    ],
    "sourceIds": ["source_water_01"]
  }
]
```

7. Store Pinia et sauvegarde locale

Toutes les réponses sont conservées uniquement dans le navigateur. L'utilisateur peut reprendre sur le même appareil. Aucun nom, e-mail ou profil n'est nécessaire.

```
import { defineStore } from 'pinia'

export const useAssessmentStore = defineStore('assessment', {
  state: () => ({
```

```

version: '1.0.0',
currentIndex: 0,
answers: {} as Record<string, string>,
household: {
  adults: 1,
  children: 0,
  elderly: 0,
  pets: 0,
  specialNeeds: false
},
checklist: {} as Record<string, boolean>,
completedAt: null as string | null
}),
actions: {
  answer(questionId: string, answerId: string) {
    this.answers[questionId] = answerId
    this.persist()
  },
  persist() {
    localStorage.setItem('resilience976-assessment', JSON.stringify(this.$state))
  },
  restore() {
    const raw = localStorage.getItem('resilience976-assessment')
    if (raw) this.$patch(JSON.parse(raw))
  },
  reset() {
    localStorage.removeItem('resilience976-assessment')
    this.$reset()
  }
}
})

```

7.1 Règles de stockage

- Utiliser une clé unique et versionnée
- Ne pas stocker de données médicales détaillées
- Ne jamais stocker d'adresse complète
- Prévoir un bouton Effacer mes données
- Restaurer automatiquement au démarrage
- Réinitialiser les données si le format change de manière incompatible

8. Construire le diagnostic

8.1 Découpage recommandé

Étape	Domaine	Exemples
1	Foyer	Nombre de personnes, enfants, dépendances, animaux
2	Logement	Type de logement, extérieur, ouvertures, zone de repli
3	Eau et alimentation	Réserves, conservation, besoins particuliers
4	Énergie et communication	Lampes, batteries, radio, recharge
5	Santé et documents	Traitements essentiels, copies protégées
6	Comportements	Alertes, consignes, avant/pendant/après

8.2 Composant QuestionCard.vue

```

<script setup lang="ts">
import type { Question } from '@models/question'

defineProps<{ question: Question; modelValue?: string }>()
const emit = defineEmits<{ 'update:modelValue': [value: string] }>()
</script>

<template>

```

```

<fieldset class="question-card">
  <legend>{{ question.text }}</legend>
  <p v-if="question.help" class="help">{{ question.help }}</p>

  <label v-for="answer in question.answers" :key="answer.id">
    <input
      type="radio"
      :name="question.id"
      :value="answer.id"
      :checked="modelValue === answer.id"
      @change="emit('update:modelValue', answer.id)"
    />
    <span>{{ answer.label }}</span>
  </label>
</fieldset>
</template>

```

8.3 Règles UX

- Une question principale par écran
- Progression visible : étape et pourcentage
- Boutons Précédent et Continuer
- Continuer désactivé si réponse obligatoire absente
- Sauvegarde après chaque réponse
- Aucune limite de temps
- Focus clavier déplacé sur le titre de la nouvelle question
- Messages simples et non culpabilisants

9. Moteur de scoring

Le moteur doit produire exactement le même résultat à réponses identiques. Il doit être isolé dans un service pur, sans dépendance à l'interface.

```

const criticalityFactor = {
  low: 1,
  medium: 1.25,
  high: 1.6,
  critical: 2
} as const

export function calculateAssessment(questions, answers) {
  const domains = new Map()
  const priorities = []

  for (const question of questions) {
    const answerId = answers[question.id]
    const option = question.answers.find(a => a.id === answerId)
    if (!option) continue

    const weighted = option.score * question.weight
    const current = domains.get(question.domain) ?? { sum: 0, weight: 0 }
    current.sum += weighted
    current.weight += question.weight
    domains.set(question.domain, current)

    const priority = (100 - option.score)
      * question.weight
      * criticalityFactor[question.criticality]

    if (option.score < 100) {
      priorities.push({ questionId: question.id, priority })
    }
  }

  const domainScores = [...domains.entries()].map(([id, value]) => ({
    id,

```

```

    score: Math.round(value.sum / value.weight)
  })))

const globalScore = Math.round(
  domainScores.reduce((sum, d) => sum + d.score, 0) / domainScores.length
)

return {
  globalScore,
  domainScores,
  priorities: priorities.sort((a, b) => b.priority - a.priority)
}
}

```

9.1 Niveaux de restitution

Score	Niveau	Message
0-39	Préparation insuffisante	Plusieurs éléments essentiels doivent être traités en priorité.
40-59	Préparation fragile	Des bases existent, mais des vulnérabilités importantes subsistent.
60-79	Bien préparé	Le foyer dispose d'une base solide à renforcer.
80-100	Très bien préparé	Maintenir et vérifier régulièrement les bonnes pratiques.

9.2 Tests unitaires indispensables

- Même réponse = même score
- Une réponse critique insuffisante remonte dans les priorités
- Les questions non répondues ne provoquent pas de division par zéro
- Les scores restent compris entre 0 et 100
- Les domaines sont calculés séparément
- Les poids modifient réellement le résultat

10. Génération des recommandations

9. Calculer les priorités de chaque réponse non optimale.
10. Récupérer les actions associées aux questions prioritaires.
11. Dédupliquer les actions ayant le même identifiant.
12. Trier par criticité, poids et écart de score.
13. Afficher au maximum 3 actions immédiates.
14. Afficher au maximum 5 actions à réaliser sous 7 jours.
15. Placer les autres actions dans la section À maintenir.

```

export function buildActionPlan(questions, answers, actions) {
  const scores = new Map<string, number>()

  for (const question of questions) {
    const option = question.answers.find(a => a.id === answers[question.id])
    if (!option || option.score === 100) continue

    const value = (100 - option.score)
      * question.weight
      * criticalityFactor[question.criticality]

    for (const actionId of question.actionIds) {
      scores.set(actionId, Math.max(scores.get(actionId) ?? 0, value))
    }
  }

  return [...scores.entries()]
    .map(([id, priority]) => ({ ...actions.find(a => a.id === id), priority }))
    .filter(Boolean)
}

```



```

    .sort((a, b) => b.priority - a.priority)
  }

```

11. Checklist et kit d'urgence

11.1 Checklist

- Cases interactives
- Progression en pourcentage
- Catégories distinctes
- Sauvegarde locale
- Possibilité d'ajouter un élément personnel local
- Bouton Réinitialiser
- Impression lisible en noir et blanc

11.2 Kit personnalisé

Le kit est construit à partir d'éléments de base et de règles conditionnelles. Les quantités exactes doivent rester éditoriales et validées : elles ne doivent pas être inventées par le développeur.

```

[
  {
    "id": "kit_base_radio",
    "label": "Moyen de recevoir les informations officielles",
    "category": "communication",
    "conditions": []
  },
  {
    "id": "kit_children",
    "label": "Éléments adaptés aux enfants",
    "category": "household",
    "conditions": [{ "field": "children", "operator": ">", "value": 0 }]
  },
  {
    "id": "kit_pets",
    "label": "Nécessaire pour les animaux",
    "category": "household",
    "conditions": [{ "field": "pets", "operator": ">", "value": 0 }]
  }
]

```

12. Page de résultats

- Score global clairement affiché
- Niveau textuel en plus de la couleur
- Graphique ou barres par domaine
- Trois priorités immédiates
- Cinq actions sous sept jours
- Lien vers chaque source
- Boutons Checklist, Kit et Télécharger le PDF
- Bouton Refaire le diagnostic
- Avertissement : outil de sensibilisation, pas service d'urgence

12.1 Ordre visuel conseillé

16. Résumé du résultat
17. Message principal
18. Priorités immédiates
19. Scores par domaine
20. Plan sous sept jours
21. Checklist et kit
22. Sources et avertissement
23. Export PDF

13. Génération du PDF côté navigateur

Le PDF doit être généré sur l'appareil afin de ne pas envoyer les réponses à un serveur. Il contient uniquement les données volontairement saisies et le résultat calculé.

```
import jsPDF from 'jspdf'

export function generateAssessmentPdf(result, actions, checklist, sources) {
  const pdf = new jsPDF({ unit: 'mm', format: 'a4' })
  let y = 18

  pdf.setFontSize(20)
  pdf.text('Résilience 976 - Mon Plan Résilience', 15, y)
  y += 12

  pdf.setFontSize(12)
  pdf.text(`Score global : ${result.globalScore}/100`, 15, y)
  y += 10

  pdf.setFontSize(14)
  pdf.text('Mes priorités', 15, y)
  y += 8

  actions.slice(0, 8).forEach((action, index) => {
    const lines = pdf.splitTextToSize(`${index + 1}. ${action.title}`, 175)
    pdf.text(lines, 15, y)
    y += lines.length * 6
    if (y > 275) { pdf.addPage(); y = 18 }
  })

  pdf.addPage()
  pdf.text('Checklist', 15, 18)
  // Ajouter les éléments, sources et date de version...

  pdf.save('mon-plan-resilience.pdf')
}
```

13.1 Contenu du PDF

- Titre et date
- Version du questionnaire
- Score global
- Scores par domaine
- Actions immédiates
- Actions sous sept jours
- Checklist
- Kit personnalisé
- Sources principales
- Mention de responsabilité

13.2 Contrôles PDF

- Lisibilité sur téléphone et ordinateur
- Pas de texte coupé
- Sauts de page corrects
- Impression noir et blanc
- Aucune réponse non saisie
- Aucune donnée personnelle implicite
- Nom de fichier compréhensible

14. Design system et responsive

Élément	Règle
Largeur de référence	360 px minimum

Élément	Règle
Boutons	Hauteur tactile d'au moins 44 px
Texte	16 px minimum pour le corps sur mobile
Contraste	Suffisant pour texte et contrôles
Couleur	Jamais utilisée seule pour transmettre un état
Navigation	Menu simple et accès direct au diagnostic
Formulaires	Libellé visible, erreur proche du champ
Progression	Texte + barre de progression
Cartes	Titre, explication, action explicite

14.1 Palette possible

- Bleu nuit : structure et titres
- Bleu-vert : actions positives
- Orange : repères et attention
- Rouge modéré : criticité
- Fond très clair : lisibilité

Les couleurs définitives doivent être vérifiées avec un outil de contraste. Les indicateurs critiques doivent aussi utiliser du texte et une icône.

15. Accessibilité

- HTML sémantique
- Un seul h1 par page
- Fieldset et legend pour les groupes de réponses
- Navigation complète au clavier
- Focus visible
- Erreurs annoncées clairement
- Alternative textuelle des images
- Zoom à 200 % sans perte de contenu
- Aucune limite de temps
- PDF lisible et structuré autant que possible
- Test avec lecteur d'écran sur au moins un environnement

15.1 Checklist manuelle

24. Naviguer uniquement avec Tab, Maj+Tab, Entrée et Espace.
25. Vérifier que le focus suit l'ordre visuel.
26. Tester à 200 % de zoom.
27. Tester un écran de 360 px.
28. Désactiver les images et vérifier la compréhension.
29. Tester les erreurs de formulaire.
30. Vérifier que chaque score possède un libellé textuel.

16. RGPD, confidentialité et sécurité

L'absence de compte et de serveur réduit fortement les risques, mais ne supprime pas les obligations de transparence et de sécurité.

- Afficher clairement que les réponses restent sur l'appareil
- Prévoir un bouton d'effacement local
- Ne pas charger de traceurs non nécessaires
- Privilégier des statistiques sans cookie ou ne rien mesurer au départ
- Ne pas intégrer de secret dans le front-end
- Ne pas présenter le service comme une alerte officielle
- Utiliser HTTPS
- Mettre à jour les dépendances
- Ajouter une Content Security Policy lorsque l'hébergement le permet
- Éviter les bibliothèques externes inutiles

16.1 Mention publique recommandée

Résilience 976 est un outil de sensibilisation et de préparation. Il ne fournit pas d'alerte en temps réel, ne recueille pas les demandes de secours et ne remplace pas les consignes diffusées par les autorités compétentes.

17. Préparation et validation des contenus

Le développeur ne doit pas inventer les consignes de sécurité. Les questions, scores, recommandations, quantités et sources doivent être fournis ou validés par un référent métier.

Objet	Responsable	Validation
Questions	Référent risques + porteur	Formulation simple et absence d'ambiguïté
Scores	Porteur + référent	Poids, criticité et seuils
Actions	Référent risques	Exactitude et faisabilité
Kit	Référent risques	Quantités et recommandations officielles
Sources	Responsable éditorial	Organisme, date, URL, droits
Mentions légales	Porteur	Responsabilité et confidentialité

17.1 Versionner les contenus

- Ajouter assessmentVersion dans chaque diagnostic
- Conserver la date de révision
- Associer chaque recommandation à une source
- Noter toute modification du barème
- Ne jamais mélanger les résultats de deux versions dans des statistiques

18. Tests

Type	Ce qu'il faut tester
Unitaire	Scoring, priorités, déduplication, kit conditionnel
Composants	QuestionCard, navigation, progression, erreurs
Parcours complet	Accueil vers PDF sans compte
Compatibilité	Chrome Android, Safari iPhone, Firefox, Edge
Accessibilité	Clavier, focus, zoom, lecteur d'écran
Performance	Réseau lent, images, poids JavaScript
Éditorial	Sources, dates, textes, liens
PDF	Mobile, desktop, impression

18.1 Exemple Vitest

```
import { describe, expect, it } from 'vitest'
import { calculateAssessment } from '@services/scoring.service'

describe('calculateAssessment', () => {
  it('retourne le même score pour les mêmes réponses', () => {
    const first = calculateAssessment(questions, answers)
    const second = calculateAssessment(questions, answers)
    expect(first).toEqual(second)
  })

  it('place une réponse critique insuffisante dans les priorités', () => {
    const result = calculateAssessment(questions, criticalAnswers)
    expect(result.priorities[0].questionId).toBe('water_01')
  })
})
```

19. Performance et mode connexion limitée

- Chargement différé des pages avec import dynamique
- Compression des images
- Pas de vidéo préchargée
- JSON de taille raisonnable
- Polices système si possible

- Pas de bibliothèque graphique lourde pour un simple score
- Cache PWA optionnel après stabilisation
- Aucun appel réseau nécessaire pendant le diagnostic

19.1 PWA optionnelle

Une PWA peut être ajoutée après le MVP afin de mettre en cache l'accueil, les questions et les ressources essentielles. Elle ne doit pas retarder le lancement initial.

20. Déploiement

20.1 Build de production

```
npm run test
npm run build
npm run preview
```

20.2 Netlify ou Vercel

- Connecter le dépôt Git
- Commande de build : `npm run build`
- Dossier publié : `dist`
- Activer HTTPS
- Configurer la redirection SPA vers `index.html`
- Ajouter le domaine final
- Tester toutes les routes directement

20.3 Fichier de redirection Netlify

```
/* /index.html 200
```

20.4 Variables d'environnement

```
VITE_APP_NAME=Résilience 976
VITE_ASSESSMENT_VERSION=1.0.0
VITE_PUBLIC_BASE_URL=https://exemple.fr
```

Aucune clé secrète ne doit être placée dans une variable `VITE_` : tout ce qui est exposé au front peut être lu par l'utilisateur.

21. Git et CI/CD

- Branche main protégée
- Une branche par fonctionnalité
- Pull request avant fusion
- Tests automatiques
- Build automatique
- Déploiement de prévisualisation
- Tag de version pour chaque publication

```
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
```

```
- run: npm run test
- run: npm run build
```

22. Planning réaliste pour un développeur seul

Semaine	Travail principal	Résultat
1	Cadrage, validation des contenus, installation, routes, design system	Projet navigable
2	Modèles JSON, store Pinia, diagnostic et sauvegarde locale	Questionnaire fonctionnel
3	Scoring, résultats et recommandations	Cœur métier terminé
4	Checklist, kit, ressources et PDF	MVP complet
5	Accessibilité, responsive, tests, corrections et déploiement	Version de production

Ce planning suppose que les contenus sont disponibles et validés. La création des consignes, des questions et du barème peut prendre autant de temps que le développement.

23. Critères d'acceptation du MVP

ID	Critère
MVP-01	Le diagnostic fonctionne sans compte sur un écran de 360 px.
MVP-02	La progression est reprise sur le même appareil.
MVP-03	Le score est reproductible à réponses identiques.
MVP-04	Une faiblesse critique apparaît dans les priorités.
MVP-05	Le plan ne contient aucune action dupliquée.
MVP-06	La checklist est sauvegardée localement.
MVP-07	Le kit s'adapte à la composition du foyer.
MVP-08	Le PDF est lisible et imprimable.
MVP-09	Chaque recommandation sensible possède une source.
MVP-10	L'utilisateur peut effacer toutes ses données locales.
MVP-11	Le parcours est utilisable au clavier.
MVP-12	Aucune fonction ne prétend fournir une alerte en temps réel.

24. Checklist avant mise en ligne

- Nom public et domaine validés
- Questions et barème validés
- Sources officielles vérifiées
- Mentions légales publiées
- Politique de confidentialité publiée
- Avertissement de responsabilité visible
- Toutes les routes testées
- PDF testé sur Android et iPhone
- Tests clavier réalisés
- Contrastes vérifiés
- Aucun lien cassé
- Build de production sans erreur
- HTTPS actif
- Dépôt et comptes appartenant au porteur
- Documentation d'installation à jour

25. Évolutions après le MVP

Version	Évolution	Condition
V1.1	Quiz simple	Après stabilisation du diagnostic
V1.2	Analytics anonymes	Après validation de la politique de mesure
V1.3	PWA et cache hors ligne	Après tests de performance
V2	Campagnes et statistiques agrégées	Nécessite un back-end minimal
V2	CMS éditorial	Lorsque les mises à jour deviennent fréquentes
V3	Assistant RAG sourcé	Après constitution d'un corpus validé

Version	Évolution	Condition
V3	Cartographie modérée	Après gouvernance et partenaires

26. Définition finale du projet à développer

Résilience 976 est une application web Vue.js publique, mobile-first et sans compte. Elle fonctionne principalement dans le navigateur. L'utilisateur réalise un diagnostic, reçoit un score et un plan d'actions, complète une checklist, génère un kit d'urgence adapté et télécharge un PDF. Les questions, scores, recommandations et sources sont stockés dans des fichiers JSON versionnés. Les réponses sont conservées uniquement dans localStorage et peuvent être effacées à tout moment. Aucun back-end ni base de données n'est nécessaire pour le MVP.

26.1 Commandes finales utiles

```
# Démarrer en local
npm run dev

# Lancer les tests
npm run test

# Créer la version de production
npm run build

# Vérifier la version compilée
npm run preview
```

Base documentaire : Résilience 976 - Book Produit & Spécifications Fonctionnelles et Techniques, version 1.0 du 3 août 2026. Le présent guide applique un périmètre volontairement simplifié : un seul front Vue.js, sans compte ni base de données.